Installing Hadoop

At a minimum, Hadoop necessitates an operating system that operates on a 64-bit architecture and has at least 8GB of RAM.

To begin the process of installing Hadoop, obtain the latest version from the official Apache Hadoop website. After downloading the archive, extract its contents to an appropriate directory on your computer.

The next step is to edit the configuration files, specifically *core-site.xml* and *hdfs-site.xml* for configuring HDFS, and *mapred-site.xml* for configuring the MapReduce framework. Additionally, to enable Hadoop to function correctly, several environment variables, such as HADOOP_HOME, JAVA_HOME, and PATH, need to be set.

Before using HDFS, it must be formatted with the command *bin/hdfs namenode -format.* Once Hadoop is installed and configured, it can be started by running *sbin/start-all.sh*. To verify that Hadoop is working correctly, you can run sample MapReduce jobs provided as examples.

It is crucial to acknowledge that the installation and configuration of Hadoop can be a challenging undertaking, and the specific steps required may vary depending on your system and setup. It is advisable to refer to the official Apache Hadoop documentation and consult an experienced Hadoop administrator if you experience any difficulties.

Streaming Hadoop

To stream Hadoop, follow these steps.

1. First, make sure that you have Hadoop installed and configured correctly.
2. Create a Python script that will serve as your mapper. The mapper supports key-to-value pairs and outputs intermediary key-to-value pairs.

The *mapper.py* code in Python is part of a MapReduce job, and its function is to transform input data into key-to-value pairs. In the Hadoop context, the mapper takes a portion of the data and processes it to produce key-value pairs in between. These key-to-value pairs are then sent to the reducing agent for further processing.

The mapper code typically reads data from standard input (stdin), processes it in some way, and outputs key-to-value pairs to standard output (stdout), with each pair separated by a tab (\t) character. The reducer will then aggregate these intermediate results based on the keys.

In the example I provided earlier, the *mapper.py* code reads each line of text from standard input, splits it into individual words, and then outputs each word with a count of 1 as a key-value pair. This is a common pattern in MapReduce, where the mapper produces intermediate results that can be easily aggregated by the reducer.

```
mapper.py
#!/usr/bin/python
# Format of each line is:
# date\ttime\tstore name\titem
description\tcost\tmethod of payment
# We want elements 2 (store name) and
4 (cost)
# We need to write them out to
standard output, separated by a tab
import sys

for line in sys.stdin:
    data = line.strip().split("\t")
    if len(data) == 6:
        date, time, store, item, cost,
payment = data
        print "{0}\t{1}".format(item,
cost)
```

3. To complete the task of creating a Python script that functions as a reducer, you will need to develop a code that processes intermediate key-value pairs and generates the final output. One way to do this is to use the provided code, which demonstrates how to sum the counts of each word.

The *reducer.py* code is an essential component of a MapReduce job in Python. Its primary role is to take the intermediate key-value pairs created by the mapper and aggregate them to generate the final output. In the context of Hadoop, the reducer is responsible for processing a set of key-value pairs and producing the final output.

Typically, the reducer code reads in key-value pairs from standard input (stdin) and aggregates the values for each key before outputting the results to standard output (stdout). The reducer's primary function is to combine the intermediate results created by the mapper that share the same key into a single output value.

In summary, the *reducer.py* code in Python is an essential component of a MapReduce job that aggregates intermediate key-value pairs and generates the final output. In Hadoop, the reducer takes a set of key-value pairs, combines the intermediate results created by the mapper that share the same key, and produces the final output. The reducer typically reads in key-value pairs from standard input, aggregates the values for each key, and outputs the results to standard output.

In the example I provided earlier, the *reducer.py* code reads each key-value pair from standard input, extracts the word and count, and then aggregates the counts for each word. It then outputs each word along with its total count as a key-value pair.

```
    Reducer.py
#!/usr/bin/python
import sys
oldCategory= None
totalSales = 0
# Loop around the data
# It will be in the format key\tval
```